

Project Name: Movie Recommendation System

Team# 47

Test Stage: System

Test Date: 04/12/2025

Test Case ID#: TC-MRA-01

Name(s) of Testers: Zeeshan Bombaywala

Test Description:

Validation of Movie Recommendation Algorithm requirements (REQ-1 to REQ-8), covering Cold Start, Rating Thresholds, and Content-Based Filtering logic.

Indicate where are you storing the tests (what file) and the name of the method/functions being used. Test is being stored in milestone2/testing/ReccomendationTest.java

Automated: No

Results: Partially Fail

REQ-1: The system must implement a content-based filtering recommendation algorithm.

REQ-2: The algorithm must analyze the user's rating history to determine preferences.

REQ-3: The algorithm must identify a user's preferences based on movies they have rated 4 or 5 stars.

REQ-4: The algorithm must calculate similarity scores for unwatched movies based on their similarity to the user's highly-rated movies.

REQ-5: The most important factor for the similarity calculation must be genre overlap.

REQ-6: The system must display a list of recommendations sorted by relevance (similarity-score).

REQ-7: The system must handle the situation where a user has no rating history when they are a new user.

REQ-8: Popularity must be determined by sorting all movies in

descending order by
avg_rating, using num_ratings as a tie breaker for movies with
identical average
Ratings.

Preconditions for Test:

- movies.txt must be present in the data/ directory and contain valid movie records.
- my_ratings.txt should be empty or non-existent to simulate a "New User" (Cold Start) initially.
- The DataManager must successfully load the movie library.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Test Cold Start & Popularity (REQ-7, REQ-8)	User History: Empty	System returns 10 movies sorted by Average Rating in Descending order (e.g., The Godfather 4.8 first).	System returned movies in Ascending order (ex. The Terminator 4.3 first before 4.2).	FAIL. Error in output of getPopularMovies (See Note 1).
2	Test Rating Threshold (REQ-2, REQ-3)	MovieID: 5 (Forrest Gump) Rating: 1	The system ignores the low rating (threshold < 4) and falls back to Popular Movies (Descending).	The system returned Popular Movies, but in Ascending order.	Partial FAIL. The threshold logic worked (it fell back to popular), but failed due to the sorting bug in Step 1. Will be automatically resolved if sorting is fixed.
3	Test Content-Based Similarity (REQ-1, 4, 5, 6)	MovieID: 7 (The Matrix). Rating: 5	System returns unwatched movies with high Genre Overlap (Sci-Fi/Action) sorted by similarity score.	System returned: 1. The Incredibles, 2. How to Train Your Dragon, 3. The Terminator. (All Action/Sci-Fi).	PASS. The core recommendation engine and genre weighting are functioning correctly.

Post condition(s) for Test: The my_ratings.txt file contains the test ratings added during execution (ID 5: 1-star, ID 7: 5-stars).

Relevant Notes / Bug Report

Note 1: Known Issue (BUG-001) - Inverse Sort Order

- **Observation:** The "Popular Movies" list is sorting from Lowest Rated to Highest Rated. Even though the sorting logic does Highest Rated to Lowest Rated.
- **Investigation:** The source code logic in DataManager.java appears mathematically correct for descending order (Comparison logic: $m2 > m1$).
- **Conclusion:** The failure can be due to an old build artifact (the runtime executing an older version of the compiled class file) rather than a logic error in the source code. Even though I've tested recompiling several times, after deleting all compiled .class files. The issue persists on my device.
- **Mitigation:** This bug only affects the "New User" experience. The core assignment requirement, Content-Based Filtering (REQ-1 to REQ-6), was verified as **Working** in Test Step 3. Due to time constraints a solution for this problem has not been worked out, but a proposed solution is printing the popular movies in reverse order. However this just creates the illusion of correct sorting logic and is thus not implemented.